

Brownian Motion

Mohammad Tarek Al Masri, Maya Said, Hadi Zoghaib

This manuscript was compiled on June 1, 2026

Abstract—Microscopic particles suspended in liquid have been observed to randomly oscillate and "wiggle". This phenomenon is called Brownian motion. By tracking the motion of these particles and using the diffusion equation, we are able to model the random motion of the particles and derive an approximate value of the Boltzmann Constant using Einstein's diffusion equation.

CONTENTS

1	Introduction	1
2	Theory	1
3	Experimental Setup and Methodology	4
3.1	Materials	4
3.2	Procedure	4
4	Data Analysis and Results	4
4.1	Finding the diffusion coefficient from the linear relation (10)	4
4.2	Finding the diffusion coefficient from fitting a Gaussian to (6)	8
4.3	Values of k_B	9
5	Interpretation	9
6	Conclusion	9
7	Appendix	9
8	Bibliography	20

1. INTRODUCTION

In 1827, the Scottish botanist Robert Brown was investigating the process of fertilization of a certain species of plant, called *Clarkia Pulchella*. He had placed pollen grains suspended in water under a microscope when he noticed something unexpected: the grains were oscillating rapidly. This was later called Brownian motion. While this observation had been made in similar suspended particles before, Brown was the first to actually research it. He had first attributed the motion to the organic nature of the particles, but upon repeating the experiment with pollen from dead plants and other completely inorganic material, the hypothesis was proven wrong.

Many early physicists studying this phenomenon were unable to correctly pinpoint why the motion was happening. However, the development of the kinetic theory of gases in the late 1800's lead one Albert Einstein to develop his own model for the phenomenon of Brownian Motion in 1905. The model was later verified by Jean-Baptiste Perrin in 1908, and it is still in use today [5].

In this paper, we observe plastic microscopic particles suspended in water in order to study Brownian motion. Using Einstein's model along side other theoretical and numerical tools, we derive the Boltzmann constant, which relates the average kinetic energy of gas particles to the gas's thermal energy.

2. THEORY

Brownian motion occurs in the context of colloidal dispersions. This heterogeneous system consists of particles having dimensions on the order of $10\ \mu m$ (or less) dispersed in a liquid medium [1]. The particle undergoes a large number of collisions which manifests as random motion, characterized by ideal random walk behavior at thermal equilibrium. The scale of these collisions can be seen in water particles that undergo around 10^{11} collisions per

second [2]. The net movement of a particle over a long duration will average to zero since collisions are random. However, at any specific instance in time there will be a net random force F_r or momentum transfer in a random direction. This fact motivates the Langevin equation, following section 4.2 on colloidal dispersion in [1], which models the random motion of a particle

$$m\ddot{\mathbf{R}} + \xi\dot{\mathbf{R}} = F_r, \quad (1)$$

where ξ is the drag coefficient. By Stoke's law [1], we can write ξ for a spherical particle with radius a in terms of the viscosity of the medium η

$$\xi = 6\pi\eta a \quad (2)$$

Whereas, if we drop the random force term F_r , due to friction of fluid medium velocity will drop exponentially

$$v(t) = v_0 e^{-\frac{\xi}{m}t}$$

Due to the randomness of collision in any direction, the average displacement in any direction is the same. In other words, $\langle x^2 \rangle = \langle y^2 \rangle = \langle z^2 \rangle = \frac{1}{3} \langle \mathbf{R}^2 \rangle$. Hence, we can restrict our analysis to one-dimension x .

We will rewrite (1) in terms of x and rearrange. Using $\frac{dx^2}{dt} = 2x\frac{dx}{dt}$ and multiplying the equation by x we get

$$mx\frac{d^2x}{dt^2} + \xi\frac{1}{2}\frac{dx^2}{dt} = xF_r$$

Additionally, substitute $x\frac{d^2x}{dt^2} = \frac{d}{dt}\left(x\frac{dx}{dt}\right) - \left(\frac{dx}{dt}\right)^2$, rearrange, and take the average of each term

$$\frac{\xi}{2}\frac{d\langle x^2 \rangle}{dt} = \langle xF_r \rangle - m\frac{d}{dt}\left\langle x\frac{dx}{dt} \right\rangle + m\left\langle \left(\frac{dx}{dt}\right)^2 \right\rangle$$

The starting point is at zero, as such the $\langle x \rangle = 0$. The first term on RHS averages to zero since the position and the direction of the force are both random and uncorrelated. Similarly, the second term averages to zero since the velocity has no preferred direction and has no correlation with position. The third term up to a factor of $\frac{1}{2}$ is the average kinetic energy term which is equal to $\frac{1}{2}k_B T$ by the equipartition theorem. Hence

$$\frac{d\langle x^2 \rangle}{dt} = \frac{2k_B T}{\xi}$$

which we integrate to get

$$\langle x^2 \rangle = \frac{2k_B T}{\xi} t \quad (3)$$

Next, we will look at the random walk and how it satisfies the diffusion equation. Consequently, we will find the probability of a particle's undergoing Brownian motion is normal, in addition to finding another expression for $\langle x^2 \rangle$.

We will examine [3] the mathematical formulation for the random walk on a one-dimensional lattice or x-axis, the generalization to higher dimensions is straight forward. Suppose at a time t a particle is at position x . The position at time $t + \Delta t$ is either $x + \Delta x$ or $x - \Delta x$, where Δt and Δx are positive discrete time and spatial steps, respectively.

Let $P(x, t)$ be the probability of a particle being at position x at time t . Moreover, knowing the $x = x_0$ at a previous time t_0 ,

$$P(x, t|x_0, t_0) := P(x, t|x = x_0, t = t_0)^1$$

is the probability of the particle being at position x at time $t > t_0$. At each time step, there will be a distribution which describes the probabilities of transitions. In the ideal random walk case we have

$$P(x_0 + \Delta x, t_0 + \Delta t|x_0, t_0) = P(x_0 - \Delta x, t_0 + \Delta t|x_0, t_0) = \frac{1}{2} \quad (4)$$

while in the case of a drift to the right for instance $P(x_0 + \Delta x, t_0 + \Delta t|x_0, t_0) > P(x_0 - \Delta x, t_0 + \Delta t|x_0, t_0)$.

Physically, we assume that each particle, after each collision attains a constant speed v_0 in a random direction, due to isotropy, and travels for duration Δt , to cover distance $\Delta x = v_0 \Delta t$ [1]. In a physical system, velocity change will

¹Recall that $P(A|B)$ is the probability of the event A knowing that the event B is true.

be more gradual and at thermal equilibrium velocities will vary according to the Maxwell-Boltzmann distribution. However, as we will see later, the incomplete picture produces fairly accurate statistics.

Now, we will show that the ideal random walk satisfies the diffusion equation by writing the conditional probability equation and taking the continuum limit. First, we have

$$P(x, t + \Delta t) = P(x, t + \Delta t | x - \Delta x, t)P(x - \Delta x, t) + P(x, t + \Delta t | x + \Delta x, t)P(x + \Delta x, t)$$

$$P(x, t + \Delta t) = \frac{1}{2}P(x - \Delta x, t) + \frac{1}{2}P(x + \Delta x, t)$$

This equation says that the probability of arriving at x is the same coming from the left or from the right. Next, we will expand in time on the LHS up to first order correction, while we expand in space on the RHS up to second order corrections

$$P(x, t) + \Delta t \frac{\partial P(x, t)}{\partial t} = \frac{1}{2} \left(P(x, t) - \Delta x \frac{\partial P(x, t)}{\partial x} + \frac{(\Delta x)^2}{2} \frac{\partial^2 P(x, t)}{\partial x^2} \right) + \frac{1}{2} \left(P(x, t) + \Delta x \frac{\partial P(x, t)}{\partial x} + \frac{(\Delta x)^2}{2} \frac{\partial^2 P(x, t)}{\partial x^2} \right)$$

$$P(x, t) + \Delta t \frac{\partial P(x, t)}{\partial t} = P(x, t) + \frac{(\Delta x)^2}{2} \frac{\partial^2 P(x, t)}{\partial x^2}$$

Hence, we get the diffusion equation

$$\frac{\partial P}{\partial t} = D \frac{\partial^2 P}{\partial x^2}, \quad (5)$$

where $D = \frac{(\Delta x)^2}{2\Delta t}$ is the diffusion coefficient. Initially the particle begins at $x = 0$ which is equivalent to solving the particular solution for the diffusing equation with Dirac-Delta pulse at 0. The solution is the Green's function for the diffusion equation which [4] shows to be a Gaussian distribution

$$P(x, t) = \frac{1}{\sqrt{2\pi(2Dt)}} e^{-\frac{1}{2} \frac{x^2}{2Dt}} \quad (6)$$

At time t , the variance is $\sigma^2 = 2Dt$. Since $\langle x \rangle = 0^2$ it is easy to see that

$$\langle x^2 \rangle = 2Dt \quad (7)$$

Combining this result with (3), we get Einstein's formula

$$D = \frac{k_B T}{\xi} \quad (8)$$

Consequently, we also get the Stokes-Einstein equation

$$D = \frac{k_B T}{6\pi\eta a} \quad (9)$$

The Stokes-Einstein equation combines Einstein result, linking diffusion with temperature and drag of the medium, with Stoke's description of drag in a fluid.

In our data, we are observing movements in the $x - y$ plane. Recall that behavior in each direction is identical and independent. As such, we have the total average displacement is

$$\langle R^2 \rangle = \langle x^2 \rangle + \langle y^2 \rangle = \frac{4k_B T}{6\pi\eta a} = 4Dt \quad (10)$$

Due to the memorylessness of Brownian motion, the displacement Δx over any fixed time interval Δt is independent of the particle's history. Hence, rather than tracking the absolute position $x(t)$, we can instead study the displacement $\Delta x = x(t + \Delta t) - x(t)$ over a fixed lag Δt . Since the particle effectively "restarts" at each step, Δx follows the same Gaussian distribution as (6)

$$P(\Delta x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(\Delta x)^2}{2\sigma^2}} \quad (11)$$

²Recall the the variance $\sigma^2 = \langle x^2 \rangle - \langle x \rangle^2$.

where $\sigma^2 = 2D\Delta t$. This form is particularly useful experimentally: by collecting all displacements Δx across all particles and time steps for a fixed Δt , we can fit this Gaussian to extract σ^2 , and hence the diffusion coefficient D .

3. EXPERIMENTAL SETUP AND METHODOLOGY

3.1. Materials

The following materials were used for the experiment:

- Particle suspension with spherical plastic beads of radius $a = 0.53 \pm 0.07 \mu\text{m}$ (measured in data analysis section)
- Annulus-shaped stickers
- Microscope with 4×, 10×, 40×, 100× objective lenses
- Microscope slides and coverslip
- Camera (a phone camera was used)

3.2. Procedure

1. We first place an annulus-shaped sticker on a slide, then apply a few drops of the particle suspension inside the hole of the annulus, then cover the hole with a coverslip
2. We place the slide under the microscope, and begin by focusing on the 'inner edge' of the annulus until it appears clear. We do this with increasing magnification, going from the 4× objective lens, then 10×, 40×, and finally 100×
3. We allow the particle suspension to stabilize, then we record 20 seconds of particle motion under the microscope
4. We then move to a different region under the microscope, and step 3 is repeated
5. Step 4 is repeated until we have recorded 21 particles in motion
6. After that is all done, we place the 600/mm grating under the microscope, and film it

4. DATA ANALYSIS AND RESULTS

4.1. Finding the diffusion coefficient from the linear relation (10)

We first import the video of the diffraction grating into the [Tracker Video Analysis and Modeling Tool](#). We set the scale of measurements based on the grating (we can take 10 grooves and set that to be 1/60 mm). After the scale is set, we import the videos of particles moving into the software, and we take measurements of particle radii, and average them. This size is recorded in table 1.

Material	Average Radius (μm)
Plastic Beads	0.53 ± 0.07

Table 1. Average radius of plastic beads used in the experiment.

We can now begin recording data for particle motion. The Tracker software has an auto-tracker for this purpose. The raw data is too much to put in this report, but the graphs are shown in figure 1. We note that, when we took averages of this data, we noticed a small but significant drift in both the x and y positions of the particles. This drift is more noticeable for motion on the x -axis. We tried to account for this drift by subtracting it from our data. This is also shown in 1. It turns out, however, that this drift did not affect the final results by much.

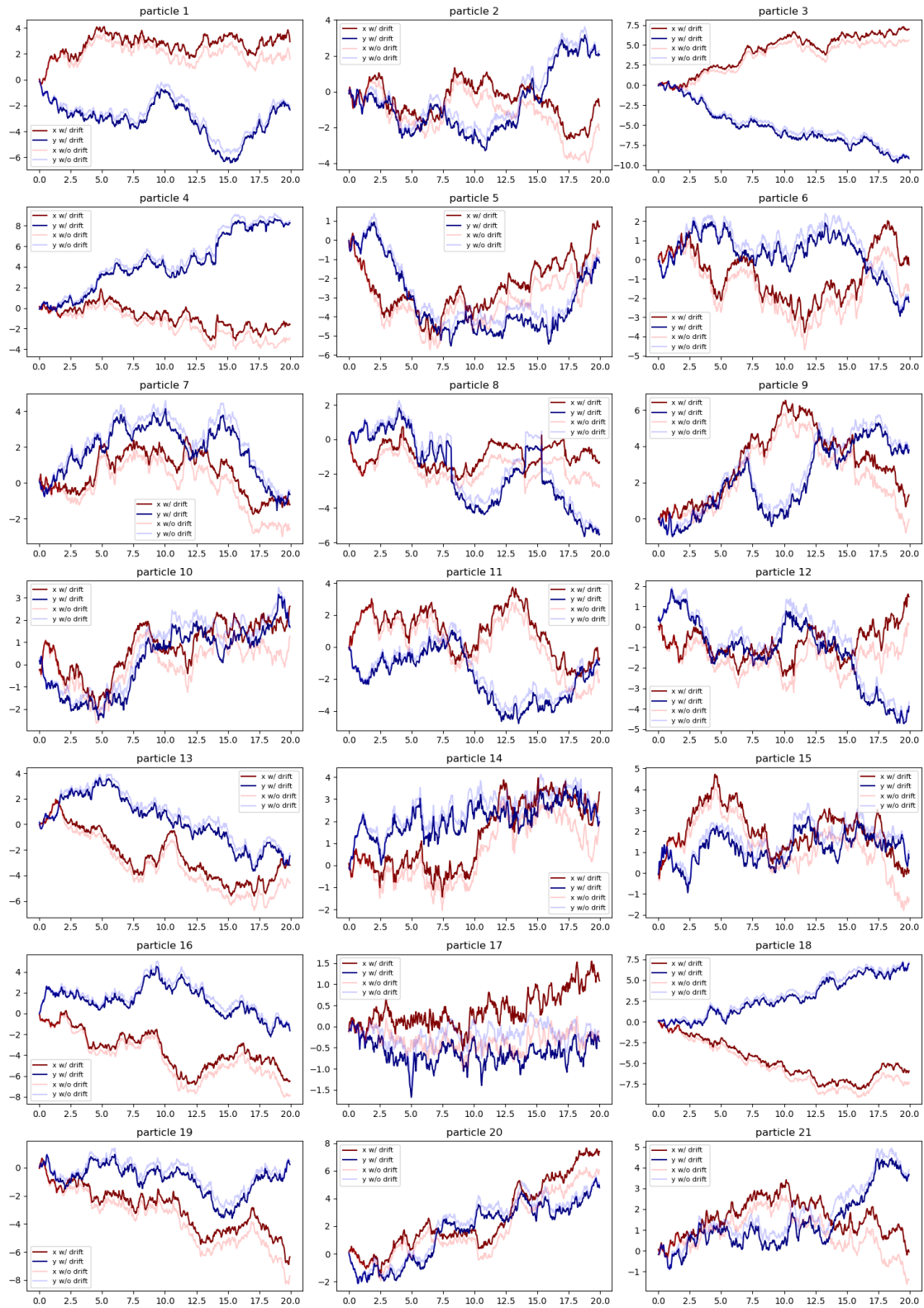


Figure 1. x and y positions of particles over time (in red and blue respectively). x and y positions of particles over time, accounting for drift (in faded red and blue, respectively)

The data for the motion of all the particles is collected in one graph for each of the x and y coordinates in figure 2, alongside the average particle drift in each direction.

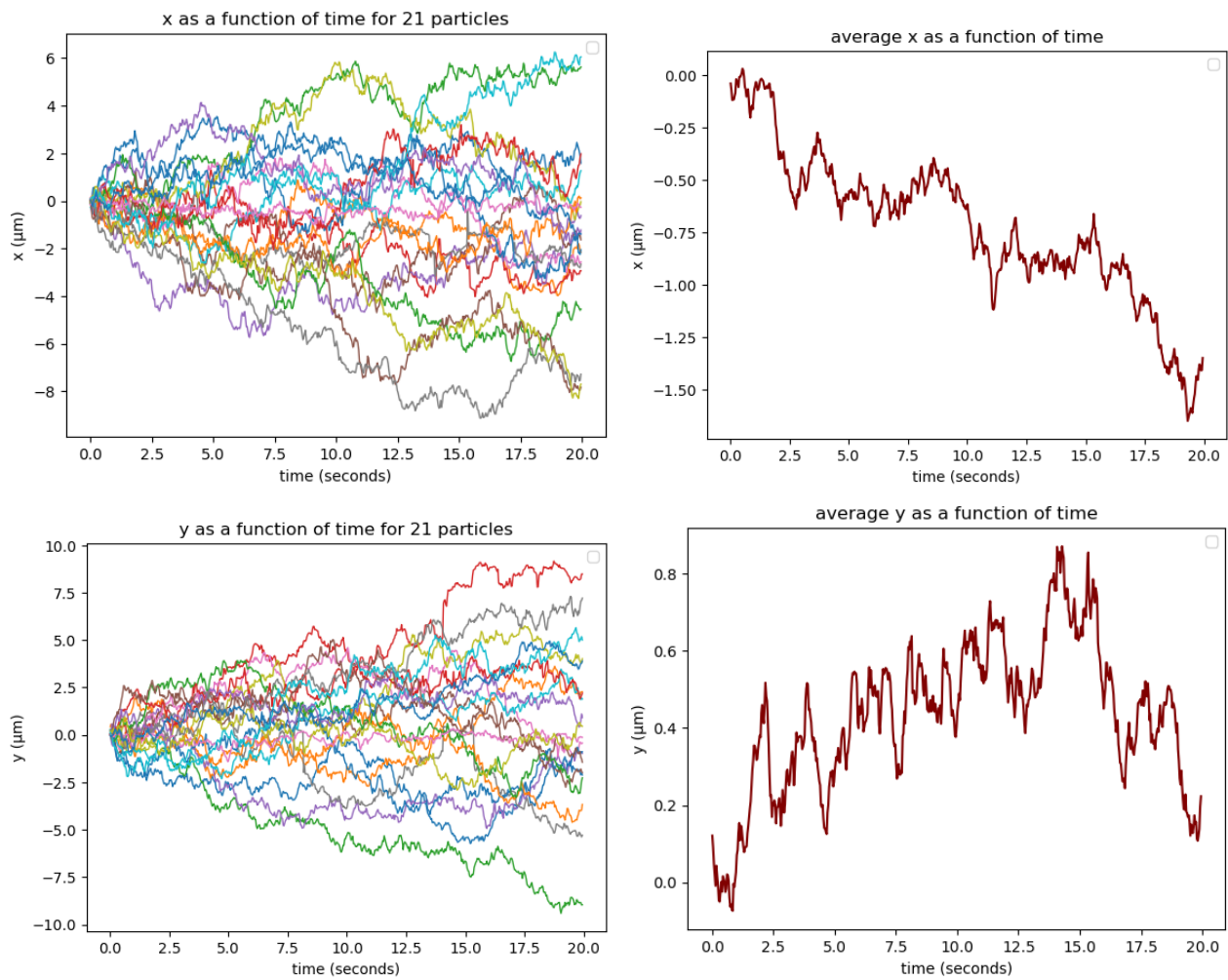


Figure 2. Data for all particles in motion (on the left) and their averages (on the right)

We square this data and obtain the graphs in figure 3.

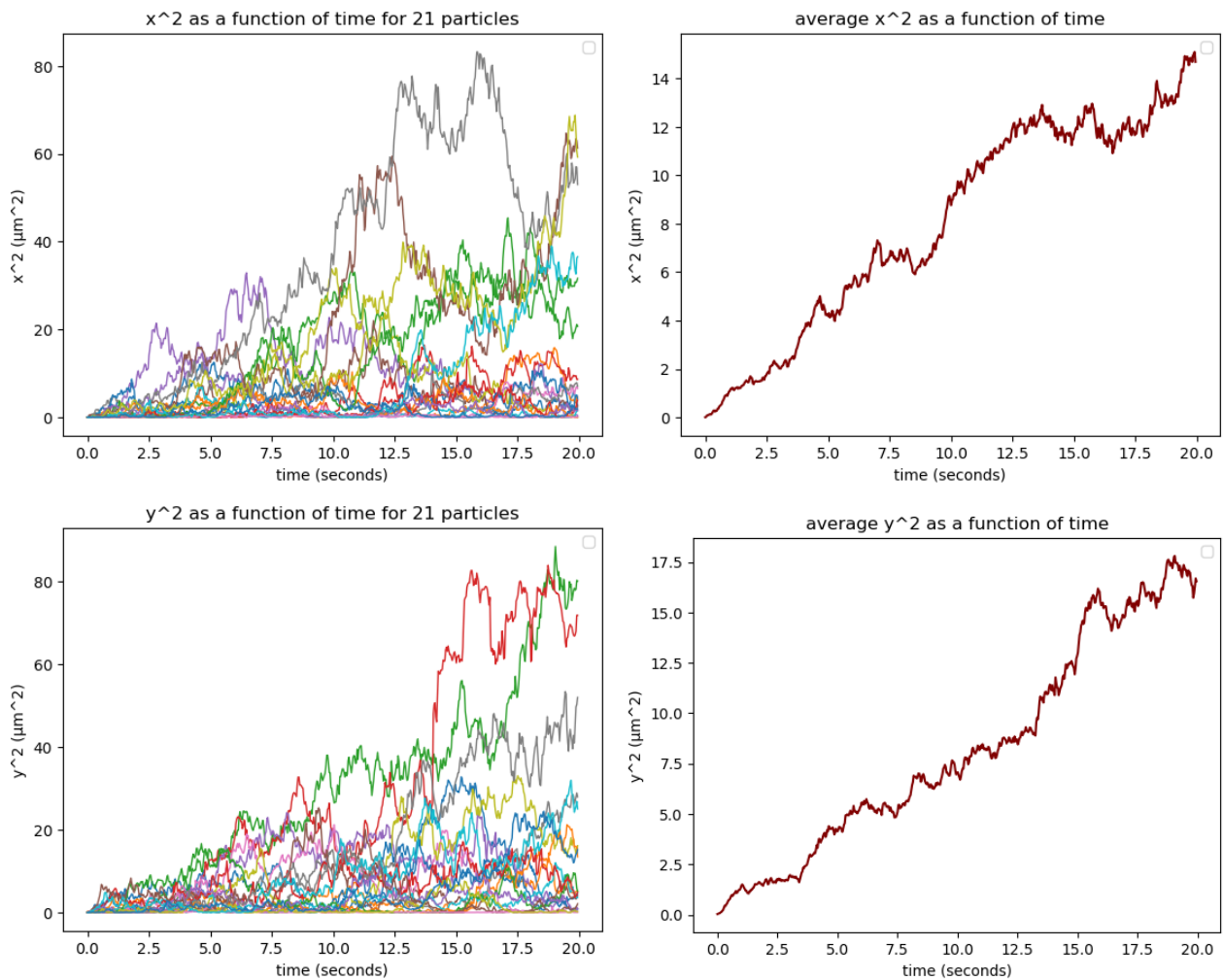


Figure 3. Square of particle displacement over time

We can now add the $\langle x^2 \rangle$ data to the $\langle y^2 \rangle$, and we obtain the graph in figure 4. We also perform a linear fit to this data, expecting equation (10) to hold.

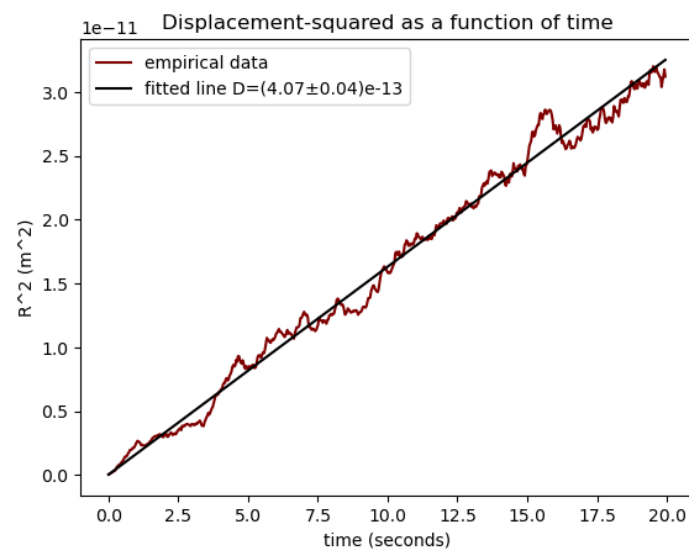


Figure 4. $\langle x^2 \rangle + \langle y^2 \rangle$ as a function of time

We find, as a result, that the diffusion constant is $D = (4.07 \pm 0.04) \cdot 10^{-13} \text{ m}^2/\text{s}$.

4.2. Finding the diffusion coefficient from fitting a Gaussian to (6)

For a given timestep Δt , we record all displacements $\Delta x, \Delta y$ that occur in time Δt in our dataset. This data is collected in the scatterplot in figure 5.

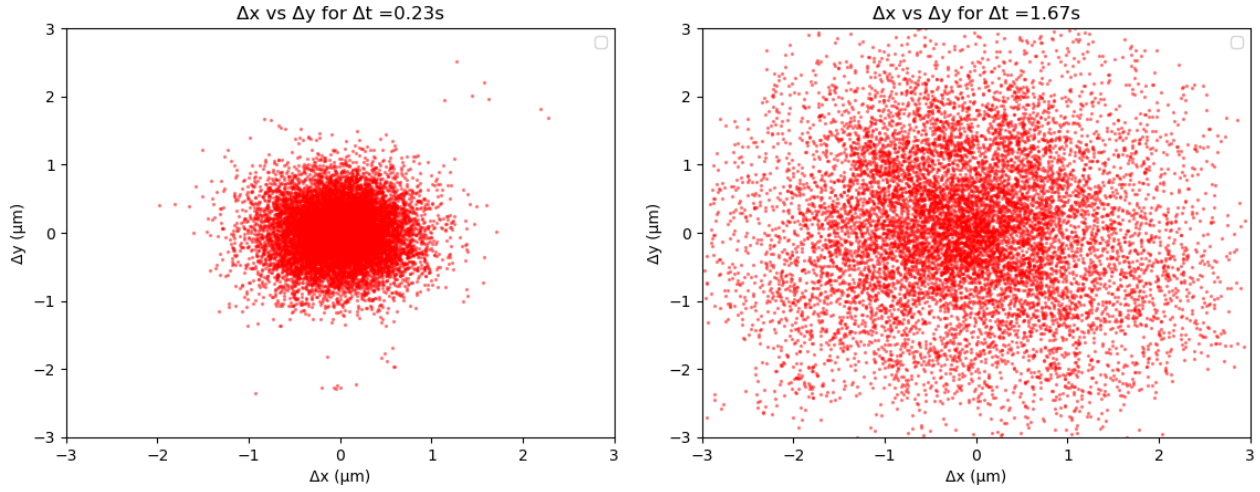


Figure 5. A scatter plot of displacements that occur for times $\Delta t = 0.23s, 1.67s$

We can place these displacements into bins, and we obtain a distribution as shown in figure 6. From the theory, we know that such a distribution is Gaussian, given by equation (11), with variance $\sigma^2 = 2D\Delta t$. Thus, we fit the distribution to a Gaussian, and solve for the diffusion constant. The fits are also shown in figure 6.

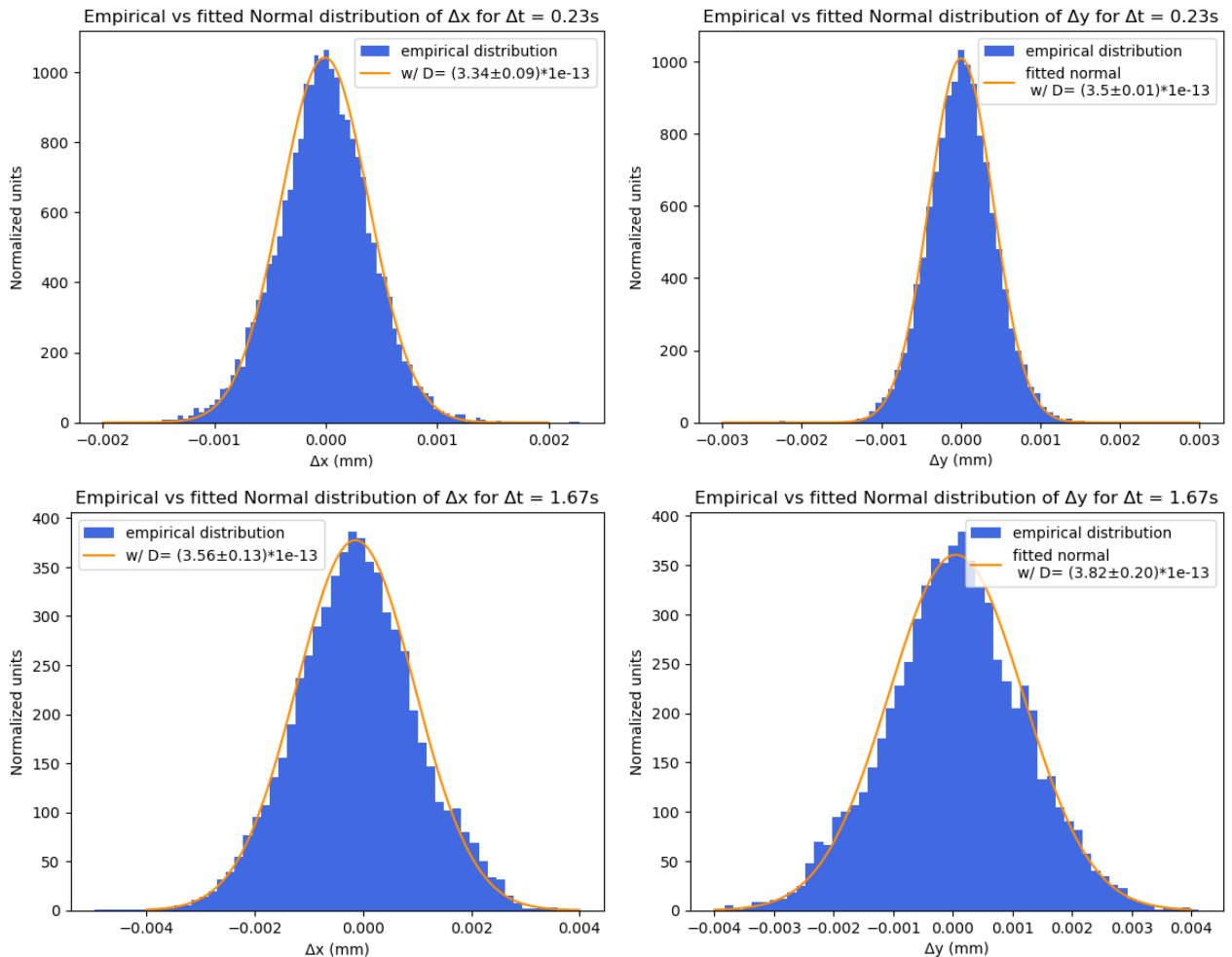


Figure 6. Binned displacements $\Delta x, \Delta y$ and their associated Gaussian fits for $\Delta t = 0.23s, 1.67s$

The derived diffusion coefficients are shown in the graphs.

4.3. Values of k_B

We can use the Stokes-Einstein equation (9) to deduce the value of Boltzmann's constant from the diffusion coefficients. This is shown in table 2.

Method	Boltzmann Constant (J/K)
Linear fit in R^2	$(1.38 \pm 0.18) \times 10^{-23}$
Normal fit for Δx (for $\Delta t = 0.23s$)	$(1.14 \pm 0.15) \times 10^{-23}$
Normal fit for Δy (for $\Delta t = 0.23s$)	$(1.19 \pm 0.15) \times 10^{-23}$
Normal fit for Δx (for $\Delta t = 1.67s$)	$(1.21 \pm 0.16) \times 10^{-23}$
Normal fit for Δy (for $\Delta t = 1.67s$)	$(1.30 \pm 0.18) \times 10^{-23}$
Literature value	1.38×10^{-23}

Table 2. Comparison of Boltzmann constant values obtained from different methods.

5. INTERPRETATION

The results obtained from all three methods are summarized in table 2. The linear fit method yielded $k_B = (1.38 \pm 0.18) \times 10^{-23}$ J/K, which is in excellent agreement with the literature value of 1.38×10^{-23} J/K. The Gaussian fitting method produced values that are slightly lower, ranging from (1.14 ± 0.15) to $(1.30 \pm 0.18) \times 10^{-23}$ J/K, but are still consistent with the literature value within two standard deviations. Notably, the results obtained for larger time lags ($\Delta t = 1.67$ s) are systematically closer to the accepted value than those at smaller lags ($\Delta t = 0.23$ s), which is consistent with the expectation that longer time intervals average over more collisions, yielding a better approximation to the Gaussian predicted by theory.

From the scatter plots in figure 5, the displacements Δx and Δy appear isotropically distributed, forming a roughly circular cloud centered at the origin. This confirms that the diffusion is mostly symmetric and that there is no significant preferred direction of motion, consistent with the assumption $\langle x^2 \rangle = \langle y^2 \rangle$ made in the theory section.

However, a small but noticeable drift was observed in the average x position of the particles over time, as seen in figure 2. While subtracting the drift had little effect on the final values of k_B , it is a source of systematic error that could bias the variance of the displacement distribution upward, leading to an overestimate of D and hence k_B .

The dominant sources of uncertainty in our experiment are the particle radius a and the fitted standard deviation σ , each contributing a relative uncertainty of approximately 13% and 3% respectively to the final value of k_B . The viscosity η was taken from the literature at $T = 293.15$ K and assumed to be exact, which introduces a potential systematic error since the actual temperature of the sample was not directly measured and small deviations from room temperature can noticeably affect η .

6. CONCLUSION

In this experiment, we studied the Brownian motion of microscopic plastic beads suspended in water. By tracking the positions of 21 particles over a time period, we were able to test theoretical predictions for diffusion and extract the Boltzmann constant through two methods.

The first method, fitting a line to $\langle R^2 \rangle = \langle x^2 \rangle + \langle y^2 \rangle$ as a function of time, gave a result in good agreement with the literature value, with a percent error of less than 1%. The second method, fitting a Gaussian distribution to the displacement histogram for a fixed Δt , consistently produced values within 17% of the literature value across all tested Δt and both spatial directions. Both methods confirm the theoretical prediction that $\langle x^2 \rangle \propto t$, validating the diffusion model.

The agreement between our experimental results and the accepted value of $k_B = 1.38 \times 10^{-23}$ J/K demonstrates that even a relatively simple optical tracking setup is sufficient to probe the statistical mechanics of microscopic systems. The primary limitations of our experiment were the uncertainty in the particle radius and the presence of a small drift, both of which could be mitigated in future experiments by using a more controlled thermal environment and a larger particle sample.

7. APPENDIX

The code used to generate all graphs above is provided below

```

import numpy as np
import scipy as sc
from scipy import constants as con
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

###Next, we transfer our x,y data for the 21 particles into 21 separate arrays

data_1 = np.array([...])
...
data_21 = np.array([...])

data_all = np.array([
    data_1, data_2, data_3, data_4, data_5, data_6, data_7,
    data_8, data_9, data_10, data_11, data_12, data_13, data_14,
    data_15, data_16, data_17, data_18, data_19, data_20, data_21
])
time_array = np.array(data_1[:,0])

sumx_data = np.zeros( 599)
sumx2_data = np.zeros(599)
sumy_data = np.zeros( 599)
sumy2_data = np.zeros(599)

sumx2_data = data_1[:,1]**2 + data_2[:,1]**2+ data_3[:,1]**2+ data_4[:,1]**2+ data_5[:,1]**2+
    ↪ data_6[:,1]**2+ data_7[:,1]**2 + data_8[:,1]**2+ data_9[:,1]**2+ data_10[:,1]**2+
    ↪ data_11[:,1]**2+ data_12[:,1]**2+ data_13[:,1]**2 + data_14[:,1]**2+ data_15[:,1]**2+
    ↪ data_16[:,1]**2+ data_17[:,1]**2+ data_18[:,1]**2+ data_19[:,1]**2+ data_20[:,1]**2+
    ↪ data_21[:,1]**2
sumy2_data = data_1[:,2]**2 + data_2[:,2]**2+ data_3[:,2]**2+ data_4[:,2]**2+ data_5[:,2]**2+
    ↪ data_6[:,2]**2+ data_7[:,2]**2 + data_8[:,2]**2+ data_9[:,2]**2+ data_10[:,2]**2+
    ↪ data_11[:,2]**2+ data_12[:,2]**2+ data_13[:,2]**2 + data_14[:,2]**2+ data_15[:,2]**2+
    ↪ data_16[:,2]**2+ data_17[:,2]**2+ data_18[:,2]**2+ data_19[:,2]**2+ data_20[:,2]**2+
    ↪ data_21[:,2]**2

sumx_data = data_1[:,1] + data_2[:,1]+ data_3[:,1]+ data_4[:,1]+ data_5[:,1]+ data_6[:,1]+
    ↪ data_7[:,1] + data_8[:,1]+ data_9[:,1]+ data_10[:,1]+ data_11[:,1]+ data_12[:,1]+
    ↪ data_13[:,1] + data_14[:,1]+ data_15[:,1]+ data_16[:,1]+ data_17[:,1]+ data_18[:,1]+
    ↪ data_19[:,1]+ data_20[:,1]+ data_21[:,1]
sumy_data = data_1[:,2] + data_2[:,2]+ data_3[:,2]+ data_4[:,2]+ data_5[:,2]+ data_6[:,2]+
    ↪ data_7[:,2] + data_8[:,2]+ data_9[:,2]+ data_10[:,2]+ data_11[:,2]+ data_12[:,2]+
    ↪ data_13[:,2] + data_14[:,2]+ data_15[:,2]+ data_16[:,2]+ data_17[:,2]+ data_18[:,2]+
    ↪ data_19[:,2]+ data_20[:,2]+ data_21[:,2]

sumx_data = sumx_data/21
sumx2_data = sumx2_data/21

x_error = np.sqrt((1/21)*((sumx_data - data_1[:,1])**2 + (sumx_data - data_2[:,1])**2+ (
    ↪ sumx_data - data_3[:,1])**2+ (sumx_data - data_4[:,1])**2+ (sumx_data - data_5[:,1])

```

```

    ↪ **2+ (sumx_data - data_6[:,1])**2+ (sumx_data - data_7[:,1])**2 + (sumx_data - data_8
    ↪[:,1])**2+ (sumx_data - data_9[:,1])**2+ (sumx_data - data_10[:,1])**2+ (sumx_data -
    ↪ data_11[:,1])**2+ (sumx_data - data_12[:,1])**2+ (sumx_data - data_13[:,1])**2 + (
    ↪ sumx_data - data_14[:,1])**2+ (sumx_data - data_15[:,1])**2+ (sumx_data - data_16
    ↪[:,1])**2+ (sumx_data - data_17[:,1])**2+ (sumx_data - data_18[:,1])**2+ (sumx_data -
    ↪ data_19[:,1])**2+ (sumx_data - data_20[:,1])**2+ (sumx_data - data_21[:,1])**2))

x2_error = np.sqrt((1/21)*((sumx2_data - data_1[:,1])**2)**2 + (sumx2_data - data_2[:,1])**2)
    ↪ **2+ (sumx2_data - data_3[:,1])**2)**2+ (sumx2_data - data_4[:,1])**2)**2+ (sumx2_data -
    ↪ data_5[:,1])**2)**2+ (sumx2_data - data_6[:,1])**2)**2+ (sumx2_data - data_7[:,1])**2)
    ↪ **2 + (sumx2_data - data_8[:,1])**2)**2+ (sumx2_data - data_9[:,1])**2)**2+ (sumx2_data -
    ↪ data_10[:,1])**2)**2+ (sumx2_data - data_11[:,1])**2)**2+ (sumx2_data - data_12
    ↪[:,1])**2)**2+ (sumx2_data - data_13[:,1])**2)**2 + (sumx2_data - data_14[:,1])**2)**2+ (
    ↪ sumx2_data - data_15[:,1])**2)**2+ (sumx2_data - data_16[:,1])**2)**2+ (sumx2_data -
    ↪ data_17[:,1])**2)**2+ (sumx2_data - data_18[:,1])**2)**2+ (sumx2_data - data_19[:,1])**2)
    ↪ **2+ (sumx2_data - data_20[:,1])**2)**2+ (sumx2_data - data_21[:,1])**2)**2))

sumy_data = sumy_data/21
sumy2_data = sumy2_data/21

y_error = np.sqrt((1/21)*((sumy_data - data_1[:,2])**2 + (sumy_data - data_2[:,2])**2+ (
    ↪ sumy_data - data_3[:,2])**2+ (sumy_data - data_4[:,2])**2+ (sumy_data - data_5[:,2])
    ↪ **2+ (sumy_data - data_6[:,2])**2+ (sumy_data - data_7[:,2])**2 + (sumy_data - data_8
    ↪[:,2])**2+ (sumy_data - data_9[:,2])**2+ (sumy_data - data_10[:,2])**2+ (sumy_data -
    ↪ data_11[:,2])**2+ (sumy_data - data_12[:,2])**2+ (sumy_data - data_13[:,2])**2 + (
    ↪ sumy_data - data_14[:,2])**2+ (sumy_data - data_15[:,2])**2+ (sumy_data - data_16
    ↪[:,2])**2+ (sumy_data - data_17[:,2])**2+ (sumy_data - data_18[:,2])**2+ (sumy_data -
    ↪ data_19[:,2])**2+ (sumy_data - data_20[:,2])**2+ (sumy_data - data_21[:,2])**2))

y2_error = np.sqrt((1/21)*((sumy2_data - data_1[:,2])**2)**2 + (sumy2_data - data_2[:,2])**2)
    ↪ **2+ (sumy2_data - data_3[:,2])**2)**2+ (sumy2_data - data_4[:,2])**2)**2+ (sumy2_data -
    ↪ data_5[:,2])**2)**2+ (sumy2_data - data_6[:,2])**2)**2+ (sumy2_data - data_7[:,2])**2)
    ↪ **2 + (sumy2_data - data_8[:,2])**2)**2+ (sumy2_data - data_9[:,2])**2)**2+ (sumy2_data -
    ↪ data_10[:,2])**2)**2+ (sumy2_data - data_11[:,2])**2)**2+ (sumy2_data - data_12
    ↪[:,2])**2)**2+ (sumy2_data - data_13[:,2])**2)**2 + (sumy2_data - data_14[:,2])**2)**2+ (
    ↪ sumy2_data - data_15[:,2])**2)**2+ (sumy2_data - data_16[:,2])**2)**2+ (sumy2_data -
    ↪ data_17[:,2])**2)**2+ (sumy2_data - data_18[:,2])**2)**2+ (sumy2_data - data_19[:,2])**2)
    ↪ **2+ (sumy2_data - data_20[:,2])**2)**2+ (sumy2_data - data_21[:,2])**2)**2))

R2_data = sumx2_data+sumy2_data
R2_error = x2_error+y2_error

#####

diameters = np.array([1.019e-3, 1.03e-3, 0.9795e-3,1.061e-3, 1.134e-3, 0.9485e-3, 1.107e-3,
    ↪ 1.056e-3, 1.181e-3, 1.065e-3])

a = np.mean(diameters)/2*10**(-3)
a_error = 2*np.std(diameters*10**(-3))/2)
print(a)
print(a_error)
print(a_error/a)

```

```
#####

R2_data_scaled = R2_data*(10**(-6))

t = 293.15
v = 1002*(10**(-6))

def line(x,a,b):
    return a*x+b

popt, pcov = curve_fit(line, time_array, R2_data_scaled, sigma = R2_error)
print(popt)

slope = pop[0]
slope_error = 2*np.sqrt(pcov[0,0])
print(slope)
print(slope_error)

print(slope_error/slope)

#####
k = ((6*np.pi*a*v)/(t*4))*slope

k_error = k*np.sqrt((slope_error/slope)**2 + (a_error/a)**2)
print(k)
print(k_error)
print(sc.constants.Boltzmann)
#####
print(slope/4)
print(slope_error/4)
plt.plot(time_array, R2_data_scaled, label = 'empirical data', color = 'maroon')
plt.plot(time_array, line(time_array, pop[0],pop[1]), label = f'fitted line D=({(slope
    ↪ /4*10**(13)):.2f} {(slope_error/4*10**(13)):.2f})e-13', color = 'black')

plt.xlabel('time (seconds)')
plt.ylabel('R^2 (m^2)')

plt.title("Displacement-squared as a function of time")

plt.legend()

#####
plt.plot(time_array, sumx_data*10**(3), color = 'maroon')
plt.xlabel('time (seconds)')
plt.ylabel('x (m)')

plt.title("average x as a function of time")

plt.legend()
#####
plt.plot(time_array, sumy_data*10**(3), color = 'maroon')
plt.xlabel('time (seconds)')
plt.ylabel('y (m)')
```

```

plt.title("average y as a function of time")

plt.legend()
#####
for i in np.arange(21):
    plt.plot(time_array, (data_all[i,:,1]*10**(3)), lw = 1)

plt.xlabel('time (seconds)')
plt.ylabel('x (m)')

plt.title("x as a function of time for 21 particles")

plt.legend()
#####

for i in np.arange(21):
    plt.plot(time_array, (data_all[i,:,2]*10**(3)), lw = 1)

plt.xlabel('time (seconds)')
plt.ylabel('y (m)')

plt.title("y as a function of time for 21 particles")

plt.legend()
#####

fig, axes = plt.subplots(7, 3, figsize=(15, 21))
axes = axes.flatten()

for i, data in enumerate(data_all):
    ax = axes[i]
    ax.plot(data[:, 0], data[:, 1]*10**(3)-sumx_data*10**(3), label="x w/ drift", color = '
↳ maroon')
    ax.plot(data[:, 0], data[:, 2]*10**(3)-sumy_data*10**(3), label="y w/ drift", color = '
↳ navy')
    ax.plot(data[:, 0], data[:, 1]*10**(3), label="x w/o drift", color = 'red', alpha = 0.2)
    ax.plot(data[:, 0], data[:, 2]*10**(3), label="y w/o drift", color = 'blue', alpha = 0.2)
    ax.set_title(f"particle {i+1}")
    ax.legend(fontsize=8)

# hide any unused axes if needed
for ax in axes[21:]:
    ax.axis("off")

plt.tight_layout()
plt.show()
#####

data_all = np.array([
    data_1, data_2, data_3, data_4, data_5, data_6, data_7,
    data_8, data_9, data_10, data_11, data_12, data_13, data_14,

```

```

    data_15, data_16, data_17, data_18, data_19, data_20, data_21
])

diameters = np.array([1.019e-3, 1.03e-3, 0.9795e-3, 1.061e-3, 1.134e-3, 0.9485e-3, 1.107e-3,
    ↪ 1.056e-3, 1.181e-3, 1.065e-3])

a = np.mean(diameters)/2*10**(-3)
a_error = 2*np.std(diameters*10**(-3))/2

R2_data_scaled = R2_data*(10**(-6))

t = 293.15
v = 1002*(10**(-6))

step = 7
t_step = time_array[step] - time_array[0]
diff_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diff_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,1]-data_all[i,:599-step
    ↪ ,1])

k = -20
hist, edges = np.histogram(diff_array, bins = int(np.sqrt((599-step)*21))+ k , density= True)

def normal(x, mu, var):
    return sc.stats.norm.pdf(x, loc = mu, scale = var )

bin_centers = (edges[:-1] + edges[1:]) / 2
popt2, pcov2 = curve_fit(normal, bin_centers, hist)

std = popt2[1]
std_error = 2*np.sqrt(pcov2[1,1])

interval = np.linspace(-2*1e-3, 2*1e-3, 200)

mn = np.mean(edges)
d2 = std**2/(2*t_step) *10**(-6)
d2_error = d2*2*std*std_error/(std**2)
print(d2)
k2 = (6*np.pi*a*d2*v)/(t)
k2_error = k2*np.sqrt((2*std*std_error/(std**2))**2+ (a_error/a)**2)
print(k2)
print(k2_error)

plt.hist(diff_array, bins = int(np.sqrt((599-step)*21))+k, density= True, color = 'royalblue'
    ↪ , label = 'empirical distribution')

```



```

plt.plot(interval, normal(interval, popt2[0], popt2[1]), color = 'darkorange', label = f'w/ D
    ↪ = ({d2*1e+13:.2f}*0.09)*1e-13')

plt.xlabel("x (mm)")
plt.ylabel('Normalized units')

plt.title(f"Empirical vs fitted Normal distribution of x for t = {t_step:.2f}s")
plt.legend()

#####
data_all = np.array([
    data_1, data_2, data_3, data_4, data_5, data_6, data_7,
    data_8, data_9, data_10, data_11, data_12, data_13, data_14,
    data_15, data_16, data_17, data_18, data_19, data_20, data_21
])

diameters = np.array([1.019e-3, 1.03e-3, 0.9795e-3, 1.061e-3, 1.134e-3, 0.9485e-3, 1.107e-3,
    ↪ 1.056e-3, 1.181e-3, 1.065e-3])

a = np.mean(diameters)/2*10**(-3)
a_error = 2*np.std(diameters*10**(-3))/2)

R2_data_scaled = R2_data*(10**(-6))

t = 293.15
v = 1002*(10**(-6))

step = 7

t_step = time_array[step] - time_array[0]
diff_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diff_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,2]-data_all[i,:599-step
    ↪ ,2])

k = -50
hist, edges = np.histogram(diff_array, bins = int(np.sqrt((599-step)*21))+k , density= True)

def normal(x, mu, var):
    return sc.stats.norm.pdf(x, loc = mu, scale = var )

bin_centers = (edges[:-1] + edges[1:]) / 2
popt2, pcov2 = curve_fit(normal, bin_centers, hist)

std = popt2[1]
std_error = 2*np.sqrt(pcov2[1,1])

interval = np.linspace(-3*1e-3, 3*1e-3, 200)
upperbound = sc.stats.norm.cdf(1*1e-3, loc = popt2[0], scale = popt2[1] )
lowerbound = sc.stats.norm.cdf(-1*1e-3, loc = popt2[0], scale = popt2[1] )

```

```

C = upperbound-lowerbound
print(C)
d2 = std**2/(2*t_step) *10**(-6)
d2_error = d2*2*std*std_error/(std**2)
k2 = (6*np.pi*a*d2*v)/(t)
k2_error = k2*np.sqrt((2*std*std_error/(std**2))**2+ (a_error/a)**2)

print(k2)
print(k2_error)

plt.hist(diff_array, bins = int(np.sqrt((599-step)*21))+k , density= True, color = 'royalblue'
        ↪ ', label = 'empirical distribution')

plt.plot(interval, normal(interval, popt2[0], popt2[1]), color = 'darkorange', label = f'
        ↪ fitted normal \n w/ D= ({d2*1e+13:.1f}0.01)*1e-13')

plt.xlabel("y (mm)")
plt.ylabel('Normalized units')

plt.title(f"Empirical vs fitted Normal distribution of y for t = {t_step:.2f}s")
plt.legend()
#####

diffx_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diffx_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,1]-data_all[i,:599-step
        ↪ ,1])

diffy_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diffy_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,2]-data_all[i,:599-step
        ↪ ,2])

plt.scatter(diffx_array*10**(3), diffy_array*10**(3), color = 'red', s =2 , alpha = 0.4)

plt.xlim(-3, 3)
plt.ylim(-3, 3)

plt.xlabel('x (m)')
plt.ylabel('y (m)')
plt.title(f"x vs y for t ={t_step:.2f}s")
plt.legend()
plt.show()
#####

data_all = np.array([

```

```

    data_1, data_2, data_3, data_4, data_5, data_6, data_7,
    data_8, data_9, data_10, data_11, data_12, data_13, data_14,
    data_15, data_16, data_17, data_18, data_19, data_20, data_21
])

diameters = np.array([1.019e-3, 1.03e-3, 0.9795e-3, 1.061e-3, 1.134e-3, 0.9485e-3, 1.107e-3,
    ↪ 1.056e-3, 1.181e-3, 1.065e-3])

a = np.mean(diameters)/2*10**(-3)
a_error = 2*np.std(diameters*10**(-3))/2)

R2_data_scaled = R2_data*(10**(-6))

t = 293.15
v = 1002*(10**(-6))

step = 50

t_step = time_array[step] - time_array[0]
diff_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diff_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,2]-data_all[i,:599-step
    ↪ ,2])

k = 0
hist, edges = np.histogram(diff_array, bins = int(np.sqrt((599-step)*21)/2) , density= True)

def normal(x, mu, var):
    return sc.stats.norm.pdf(x, loc = mu, scale = var )

bin_centers = (edges[:-1] + edges[1:]) / 2
popt2, pcov2 = curve_fit(normal, bin_centers, hist)

std = popt2[1]
std_error = 2*np.sqrt(pcov2[1,1])

interval = np.linspace(-4*1e-3, 4*1e-3, 200)
upperbound = sc.stats.norm.cdf(3*1e-3, loc = popt2[0], scale = popt2[1] )
lowerbound = sc.stats.norm.cdf(-3*1e-3, loc = popt2[0], scale = popt2[1] )
C = upperbound-lowerbound
print(C)
d2 = std**2/(2*t_step) *10**(-6)
d2_error = d2*2*std*std_error/(std**2)
k2 = (6*np.pi*a*d2*v)/(t)
k2_error = k2*np.sqrt((2*std*std_error/(std**2))**2+ (a_error/a)**2)

print(k2)
print(k2_error)

```

```

plt.hist(diff_array , bins = int(np.sqrt((599-step)*21)/2), density= True, color = '
    ↪ royalblue', label = 'empirical distribution')

plt.plot(interval, normal(interval, popt2[0], popt2[1]), color = 'darkorange', label = f'
    ↪ fitted normal \n w/ D= ({d2*1e+13:.2f}{d2_error*1e+13:.2f})*1e-13')

plt.xlabel("y (mm)")
plt.ylabel('Normalized units')

plt.title(f"Empirical vs fitted Normal distribution of y for t = {t_step:.2f}s")
plt.legend()

#####

data_all = np.array([
    data_1, data_2, data_3, data_4, data_5, data_6, data_7,
    data_8, data_9, data_10, data_11, data_12, data_13, data_14,
    data_15, data_16, data_17, data_18, data_19, data_20, data_21
])

diameters = np.array([1.019e-3, 1.03e-3, 0.9795e-3, 1.061e-3, 1.134e-3, 0.9485e-3, 1.107e-3,
    ↪ 1.056e-3, 1.181e-3, 1.065e-3])

a = np.mean(diameters)/2*10**(-3)
a_error = 2*np.std(diameters*10**(-3)/2)

R2_data_scaled = R2_data*(10**(-6))

t = 293.15
v = 1002*(10**(-6))

step = 50
t_step = time_array[step] - time_array[0]
diff_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diff_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,1]-data_all[i,:599-step
    ↪ ,1])

k = 0
hist, edges = np.histogram(diff_array, bins = int(np.sqrt((599-step)*21)/2) , density= True)

def normal(x, mu, var):
    return sc.stats.norm.pdf(x, loc = mu, scale = var )

bin_centers = (edges[:-1] + edges[1:]) / 2
popt2, pcov2 = curve_fit(normal, bin_centers, hist)

std = popt2[1]
std_error = 2*np.sqrt(pcov2[1,1])

```

```

interval = np.linspace(-4*1e-3, 4*1e-3, 200)

mn = np.mean(edges)
d2 = std**2/(2*t_step) *10**(-6)
d2_error = d2*2*std*std_error/(std**2)
print(d2)
k2 = (6*np.pi*a*d2*v)/(t)
k2_error = k2*np.sqrt((2*std*std_error/(std**2))**2+ (a_error/a)**2)
print(k2)
print(k2_error)

plt.hist(diff_array, bins = int(np.sqrt((599-step)*21)/2), density= True, color = 'royalblue'
        ↪ , label = 'empirical distribution')

plt.plot(interval, normal(interval, popt2[0], popt2[1]), color = 'darkorange', label = f'w/ D
        ↪ = ({d2*1e+13:.2f}{d2_error*1e+13:.2f})*1e-13')

plt.xlabel("x (mm)")
plt.ylabel('Normalized units')

plt.title(f"Empirical vs fitted Normal distribution of x for t = {t_step:.2f}s")
plt.legend()
#####

diffx_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diffx_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,1]-data_all[i,:599-step
        ↪ ,1])

diffy_array = np.zeros((599-step)*21)

for i in np.arange(21):
    diffy_array[(599-step)*(i):(599-step)*(i+1)] = (data_all[i, step:,2]-data_all[i,:599-step
        ↪ ,2])

plt.scatter(diffx_array*10**(3), diffy_array*10**(3), color = 'red', s =2 , alpha = 0.4)

plt.xlim(-3, 3)
plt.ylim(-3, 3)

plt.xlabel('x (m)')
plt.ylabel('y (m)')
plt.title(f"x vs y for t ={t_step:.2f}s")
plt.legend()
plt.show()

```

8. BIBLIOGRAPHY

- [1] R. A. L. Jones, *Soft Condensed Matter*. Oxford, UK: Oxford University Press, 2002, ISBN: 9780198505907.
- [2] P. Nakroshis, M. Amoroso, J. Legere, and C. Smith, “Measuring boltzmann’s constant using video microscopy of brownian motion”, *American Journal of Physics*, vol. 71, no. 6, pp. 568–573, 2003. DOI: [10.1119/1.1559656](https://doi.org/10.1119/1.1559656).
- [3] R. Wojnar, “Random walk, diffusion and wave equation”, *Acta Physica Polonica B*, vol. 44, no. 5, pp. 1067–1084, 2013. DOI: [10.5506/APhysPolB.44.1067](https://doi.org/10.5506/APhysPolB.44.1067).
- [4] MIT OpenCourseWare, *Lecture 6: Solving the diffusion equation*, MIT OpenCourseWare, Course 18.354J/1.062J/12.207J Nonlinear Dynamics II: Continuum Systems, Available at https://ocw.mit.edu/courses/18-354j-nonlinear-dynamics-ii-continuum-systems-spring-2015/457b849472ce32022b3c69c60fe125ad_MIT18_354JS15_Ch6.pdf, 2015.
- [5] The Editors of Encyclopaedia Britannica, *Brownian motion*, Encyclopedia Britannica, Accessed 1 May 2026, Apr. 2026. [Online]. Available: <https://www.britannica.com/science/Brownian-motion>.